T01.S03 — The C++ Game Toolchain

Topic: T01 — Real-Time Programming Foundations | **Depends on:** — **Status:** expanded | **Assignments:** A002 | **Est. theory time:** 1–2 sessions

> The question this subtopic answers: *what does a build I can trust look like, and what do I set up once so that the next two years of debugging are about game bugs rather than tooling?*
>
> Budget this tightly. Toolchain work is infinitely expandable and produces the feeling of progress without any. `A002` is capped at four hours on purpose.

Theory

U01 — CMake, enough of it

What it is. The de facto build description language for cross-platform C++. You are not learning CMake as a skill; you are learning the ~15% that produces a project that builds on Linux and Windows without edits.

Why it exists. You will link SDL3, OpenGL, ImGui, a glTF loader, and eventually your own libraries, on at least two platforms, for a decade of project lifetime. Doing that with hand-written makefiles is a job.

The mechanism. The modern (target-based, "CMake 3.15+") style. The core rule: **describe targets and their requirements; never set global flags.**

```
cmake_minimum_required(VERSION 3.21)
project(engine LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)          # -std=c++20, not gnu++20
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)  # for clangd / your editor

add_library(core STATIC src/core/time.cpp src/core/arena.cpp)
target_include_directories(core PUBLIC include)   # PUBLIC = consumers get it too
target_compile_options(core PRIVATE
    $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra -Wpedantic -Werror>
    $<$<CXX_COMPILER_ID:MSVC>:/W4 /WX>)

add_executable(game src/main.cpp)
target_link_libraries(game PRIVATE core SDL3::SDL3)
```

Three ideas do most of the work:

- **`PRIVATE`** / **`PUBLIC`** / **`INTERFACE`** — whether a requirement applies to this target, to its consumers, or only to consumers. Getting this right is what makes dependencies propagate correctly and stops include-path spaghetti.
- **Generator expressions** (`$<...>`) — per-configuration and per-compiler values, evaluated at generate time. Verbose, but the alternative is `if(MSVC)` blocks everywhere.
- **Presets** (`CMakePresets.json`) — named configurations so "debug with sanitizers" is `cmake --preset asan` rather than a remembered incantation.

Dependencies: `FetchContent` for source-buildable deps (SDL3, ImGui) is the lowest-friction option and keeps the repo self-contained. `find_package` for system libs. Vcpkg/Conan are fine and are more machinery than a solo project needs at this stage.

Where it bites. Setting `CMAKE_CXX_FLAGS` globally — it leaks into third-party targets, and then your `-Werror` fails on SDL's warnings. Always attach flags to *your* targets.

Read.

- `[CMAKEBOOK]` ch. 4 (targets), 15 (compiler/linker options), 33 (presets). Reference reading, not cover-to-cover.
- "An Introduction to Modern CMake" — https://cliutils.gitlab.io/modern-cmake/ — free, and enough on its own for this subtopic.

Check yourself.

1. When should an include directory be `PUBLIC` rather than `PRIVATE`?
2. Why does setting `-Werror` in `CMAKE_CXX_FLAGS` break a build that uses `FetchContent`?

U02 — Build configurations and what they're for

What it is. At least three build configurations, each with a job.

Why it exists. Debug builds are 5–20x slower and give you correct debugging. Release builds are fast and give you optimizer-scrambled debugging. Games need a third mode — playable speed, still debuggable — or you cannot debug anything that only manifests at real framerates.

The mechanism.

config	flags (GCC/Clang)	for
`Debug`	`-O0 -g`	Stepping through logic. Unplayably slow with real workloads.
`RelWithDebInfo`	`-O2 -g -DNDEBUG` off	**The one you live in.** Playable, profilable, mostly debuggable.
`Release`	`-O2 -DNDEBUG`	Shipping.
`ASan`	`-O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer`	Memory correctness. Run tests here, always.

Games conventionally add a fourth axis orthogonal to optimization: `DEVELOPMENT` (asserts, debug draw, console, cheats compiled in) vs `SHIPPING` (all stripped). `[GEA3]` ch. 2 calls these build *targets* as distinct from configurations, and the distinction is worth keeping.

Where it bites. Profiling or benchmarking in `Debug`. Every number is fiction. This is the single most common measurement mistake and it will recur in `T13`; kill the habit now.

Read.

- `[GEA3]` ch. 2.2 — build configurations in practice.

Check yourself.

1. Why is `RelWithDebInfo` sometimes harder to debug than `Debug`, and what specifically causes that?
2. What class of bug appears *only* in Release, and why?

U03 — Warnings, sanitizers, and asserts

What it is. The three cheap mechanisms that catch bugs before you have to think about them.

Why it exists. In a simulation, a bug's symptom appears arbitrarily far from its cause — a stale pointer written in AI code manifests as a rendering glitch six frames later. Anything that reports the error at the site of the error is worth a lot.

The mechanism.

Warnings as errors, from day one. `-Wall -Wextra -Wpedantic -Werror` (`/W4 /WX` on MSVC). Adding this to a codebase later is a day of work; starting with it is free. Worth adding beyond `-Wall -Wextra`: `-Wshadow` (catches a genuinely common class of bug), `-Wconversion` (noisy but catches float/int truncation — try it, keep it if you can stand it), `-Wold-style-cast`.

Sanitizers. These are the highest-value tools on this page.

- **ASan** — use-after-free, heap/stack buffer overflow, leaks. ~2x slowdown.
- **UBSan** — signed overflow, misaligned access, invalid casts, bad shifts. Near-free. Enable it in all non-shipping builds.
- **TSan** — data races. Not compatible with ASan; separate config. You will need it at `T13.S05` and `T11.S01`.
- **MSan** — uninitialized reads. Clang only, needs an instrumented libc++; usually more setup than it's worth solo. Valgrind covers the same ground more slowly.

Caveat worth knowing early: **custom allocators (`T03.S03`) hide bugs from ASan**, because your arena is one big valid allocation as far as ASan is concerned. ASan has manual poisoning hooks (`__asan_poison_memory_region`) for exactly this; `A007`'s rubric requires you to wire them up.

Asserts. Split them, as engines do:

- `ASSERT(x)` — development builds only, for programmer errors.
- `VERIFY(x)` — evaluates in all builds, for conditions with side effects.
- `ASSERT_SLOW(x)` — expensive invariant checks, off by default, on when hunting something.

Never `assert()` on data errors (a malformed asset file) — those need real handling. Assert on *code* errors.

Where it bites. Sanitizers being "too slow to leave on". At `-O1` ASan costs ~2x, which for a Pong-sized game is still hundreds of FPS. Leave it on until it actually hurts.

Read.

- Clang sanitizer docs — https://clang.llvm.org/docs/AddressSanitizer.html and `.../UndefinedBehaviorSanitizer.html`
- ASan manual poisoning — https://github.com/google/sanitizers/wiki/AddressSanitizerManualPoisoning
- `[GEA3]` ch. 3.2 — assertions and error handling in engines.

Check yourself.

1. Why can't ASan and TSan run in the same build?
2. What does a custom pool allocator do to ASan's ability to catch a use-after-free, and what's the fix?
3. Give an example of a condition that should be `VERIFY` and not `ASSERT`.

U04 — Debugging real-time programs

What it is. The techniques that work when the bug is at frame 4,182 and stopping the program destroys the state you need to see.

Why it exists. Breakpoint-and-step is a poor fit for a program that must keep running to reproduce the problem, and useless for anything timing-dependent.

The mechanism. In rough order of usefulness for games:

1. **Debug drawing.** A `DebugDraw::line/box/sphere/text` API with a one-frame lifetime, drawn on top of everything. This is the single most valuable debugging tool in game development, and `T05` and `T10` are near-impossible without it. Build it in `A006`, keep it forever.
2. **Conditional and data breakpoints.** `break if entity_id == 47`. GDB: `break foo.cpp:120 if id == 47`. Watchpoints (`watch expr`) catch "who is corrupting this value", which is otherwise brutal.
3. **Time control.** Pause, single-step one simulation tick, slow to 0.1x. Requires the fixed timestep from `T03.S01`. Costs an hour, saves days.
4. **Frame logging with a ring buffer.** Log the last N frames of state to a preallocated ring, dump on assert. Printf-per-frame changes the timing and floods the console; a ring buffer doesn't.
5. **State hashing.** One `uint64` per frame summarising simulation state, printed on request. Divergence between two runs is instantly bisectable to a frame. Pays off enormously at `A021` and `A047`.
6. **RenderDoc** for anything GPU (`T04` onward). A frame capture shows every draw call, its state, and its inputs. There is no equivalent technique with a CPU debugger.

Where it bites. Debugging by printf in the frame loop. It works for five minutes and then the console output is the bottleneck and the timing has changed enough that the bug moved.

Read.

- `[GEA3]` ch. 3.5 — in-game debugging facilities. Read the debug drawing section closely; it is a spec for what to build.
- GDB manual — the sections on watchpoints and conditional breakpoints.
- RenderDoc docs — https://renderdoc.org/docs/ — skim now, use from `T04`.

Check yourself.

1. Why does printf-debugging fail specifically in a real-time loop, in a way it does not in a batch program?
2. What state do you need to be able to single-step a simulation tick, and what does that require of your loop's design?

U05 — Dependencies: what to take, what to write

What it is. The policy for this roadmap, and the reasoning, so you can disagree deliberately rather than accidentally.

Why it exists. "Write it yourself" and "use a library" are both defensible; what is not defensible is not knowing which one you chose and why.

The mechanism. The rule used here: **write what the roadmap is trying to teach you; take what it isn't.**

take	why
SDL3	Window, input devices, audio device, GL context. Platform abstraction is toil, not learning. `[SDLDOCS]`
Dear ImGui	Debug UI. You would spend a month rebuilding it worse.
stb_image / stb_vorbis	Format decoding. Single-header, public domain.
cgltf / tinygltf	glTF parsing. The format is large and boring; the *use* of the data is the lesson.
glad / glew	GL function loading. Generated boilerplate.
doctest / Catch2	Test framework.

write	why
Math library	`T02` is entirely this. Using GLM here would remove the topic.
Allocators, containers you need	`T03.S03` and `T13` are about exactly this.
ECS	`T03.S04`.
Renderer	`T04`, `T08`.
Physics	`T05`.
Audio mixer	`T11`. Take the device from SDL, write the mixer.
Netcode	`T14`. Take UDP sockets, write the protocol.

The general principle for real projects, once you are past the learning phase, inverts: take everything you can, write only what differentiates your game. This roadmap's inversion is deliberate and temporary.

Where it bites. Reaching for GLM in `A003` because it is faster. It is faster, and it removes the entire point of the assignment. Conversely, writing your own image decoder because it seemed principled: that is a week on PNG's filter modes, teaching you about PNG.

Read.

- `[GEA3]` ch. 2.4 — third-party SDKs and what engines typically take.
- Browse the single-header libs at https://github.com/nothings/stb — worth knowing what exists.

Check yourself.

1. State the rule for what to write yourself in this roadmap, and give one case where you would break it.
2. Why is SDL taken but the audio mixer written, when both are "audio"?

Implementation

assignment	size	target rung	what it forces you to understand
A002	S	R2	A build you trust: warnings-as-errors, sanitizers, presets, a dependency fetched and linked, and a window that opens on a machine that isn't yours

Order and overlap. `A002` is the skeleton every subsequent assignment starts from — `A003` adds a library target to it, `A005` and `A006` build directly on its loop, and `A011` replaces its SDL renderer with OpenGL. Get the CMake structure right here and you copy it 40 times; get it wrong and you fight it 40 times.

`A002` also absorbs `A001`'s timing code as its debug overlay, which is the first instance of this roadmap's overlap principle.

If you are short on time. Non-negotiable: warnings-as-errors, an ASan preset, and `compile_commands.json`. Everything else in `A002` can be added later without pain.

Discussion Log

Open Questions

Connections

- `T01.S04` — the measurement code that goes into `A002`'s overlay.
- `T03.S03` — where U03's ASan-vs-custom-allocator caveat comes due.
- `T12.S04` — build times and iteration speed, the grown-up version of this subtopic.